PUBLIC

Code Assessment

of the Sulu Extensions XXI Smart Contracts

February 06, 2025

Produced for

enzyme

by

CHAINSECURITY

Contents

1 Executive Summary 3
2 Assessment Overview 5
3 Limitations and use of report 7
4 Terminology 8
5 Open Findings 9
6 Notes 10

1 Executive Summary

Dear all,

Thank you for trusting us to help Enzyme Foundation with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Sulu Extensions XXI according to Scope to support you in forming an opinion on their security risks.

Enzyme Foundation implements a set of helper contracts for defining price feeds.

The most critical subjects covered in our audit are functional correctness, precision of arithmetic operations. The general subjects covered are error handling and gas efficiency. Security regarding all the aforementioned subjects is high.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Sulu Extensions XXI repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	23 Jan 2025	ad2af89309175828b04174e2b3c2a647bcf5cf7f	Initial Version
2	04 Feb 2025	60640fc6c660b2254634e8417d11f304e4e90674	Fixed Typo

For the solidity smart contracts, the compiler version `0.8.19` was chosen.

The contracts below were in scope:

```
contracts/release/infrastructure/price-feeds/
    utils/PriceFeedHelpersLib.sol
    primitives/utils/
        RateAggregatorBase.sol
        AggregatorRateDeviationBase.sol
```

2.1.1 Excluded from scope

All other files are out of scope. The correct usage is out of scope. It is expected that the underlying price feeds used are standard ones (e.g. no pricefeed with high decimals/answers leading to reverts in the calculations).

2.2 System Overview

This system overview describes the latest version of the contracts as defined in the Assessment Overview.

Enzyme Foundation offers a set of helper contracts for defining more derived price feeds.

PriceFeedHelpersLib. A library implementing several helper functions for price feeds.

The functionality implemented is:

1. `convertRatePrecision`: Given a rate in a given precision, the precision is scaled to a target precision.
2. `convertRateToNewQuoteAsset`: Given two rates, base rate in A/B and quote rate in B/C, the two rates are matched to return a rate A/C. Note that given the provided base rate precision, the result will be scaled by the same factor as the quote rate.
3. `invertRate`: Given a rate A/B in a given precision, the inverse rate B/A is computed in the target precision.

4. `selectRateByToleranceDeviation`: Given two rates, an ideal and a market rate, the ideal rate is returned if the relative deviation of the market rate does not exceed a given threshold. In case the threshold is exceeded, the value returned is the market rate.
5. `parsePrecisionFromChainlinkAggregator`: Reads the scaling factor of a Chainlink-like price feed (i.e. `10**decimals`).
6. `parseRateFromChainlinkAggregator`: Reads the latest round data relevant to Enzyme from a Chainlink-like price feed (answer and timestamp). Note that a negative answer will lead to a revert.
7. `formatRateAsChainlinkAggregator`: Given an answer and a timestamp, returns the data with a zero round ID, started at timestamp and answered-in round ID.
8. `selectOldestTimestamp`: Selects the minimum of two timestamps.

RateAggregatorBase. An abstract contract implementing the Chainlink interface required by Enzyme.

The feed implements `latestRoundData` to compute a rate defined by an abstract function `baseRate` to then optionally compute a new rate by pairing the base rate with a secondary rate (i.e. converting to new quote asset) which might require inverting the secondary rate. Ultimately, the result is scaled to the target decimals `DECIMALS` which corresponds to the feed's `decimals`.

AggregatorRateDeviationBase. An abstract price feed defining a base rate

The price feed inherits RateAggregatorBase. `baseRate` is defined based on two rates:

- `idealRate`: Abstract function defining an ideal rate between two assets (e.g. stETH/ETH rate defined by the stETH contract)
- `marketRate`: The rate retrieved from a Chainlink-like price feed reflecting the market price.

The price feed returns the `idealRate` as long as the market rate's relative deviation does not exceed a given threshold. Once the threshold is exceeded, the market rate is returned.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe our findings. The findings are split into these different categories:

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

6 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

6.1 Bigger Jumps in Price Feeds

Note **Version 1**

AggregatorRateDeviationBase only returns the market rate if it deviates too much from an ideal rate. As a consequence, the price will remain constant for most of the time, not accounting for smaller changes in the market rate. However, in case of a de-peg from the ideal rate, there might be a sudden switch to the market rate, leading to a bigger jump in price compared to Chainlink price feeds.

Consider the following example:

1. A fund holds 50 ETH and 50 stETH.
2. The market and ideal price of ETH and stETH are both 1 stETH per ETH.
3. Eventually, the market price drops to 0.98 stETH per ETH which Chainlink would report.
4. The price further drops to 0.95 ETH at which point the deviation could be exceeded.

Ultimately, the price feed updates are less continuous. While there is generally the potential to arbitrage price feeds, AggregatorRateDeviationBase will amplify such possibilities. For example, a user holding 50% of shares could withdraw 50 ETH after 3 so that the funds holds only stETH. After 4, 50% of shares could be bought again with 45 ETH. Hence, an attacker could make a profit of 5 ETH. Note that such an attack could be amplified by OEV (oracle extractable value).

However, assuming eventual re-peg to the ideal rate, the stability of the AggregatorRateDeviationBase can be more suitable for asset management operations.

Finally, governance should set the maximum deviation carefully to ensure suitability of the adapted risk parameters.